Boas Praticas do ATC

Boas práticas do ATC

Estas práticas ajudam a obter resultados fiáveis no Advanced Trigonometry Calculator 2.1.9.

Comecar pequeno

Construir cálculos complexos a partir de expressões menores. Guardar resultados intermedios em variáveis ou reutilizar referências como `#0`, `#1` e seguintes.

Validar parenteses cedo

Quando uma expressão fica longa, testar primeiro os fatores internos. Isto e especialmente util em expressões racionais, produtos polinomiais e inputs do solver.

Confirmar o modo angular

Usar:

```
mode
```

antes de cálculos trigonometricos. Os modos grau, radiano e gradiano alteram o significado de comandos como `sin(30)`.

Usar a sintaxe documentada para negativos

O ATC imprime frequentemente coeficientes negativos com _, por exemplo _5. Usar a sintaxe documentada quando se introduzem listas de coeficientes negativas ou comandos que esperam esse formato.

Usar verbose resolution para debug

Ativar output diagnostico apenas quando necessario:

```
verbose resolution(1)
verbose resolution(0)
```

O verbose e util para investigar comportamento do parser, mas os cálculos normais devem geralmente correr com essa setting desligada.

Escolher o caminho certo de resolucao

Usar `solve equation(...)` para formas polinomiais e equacoes suportadas. Usar `solver(...)` para workflows numéricos de procura de raizes.

Usar TXT workflows para repeticao

Para lotes de comandos repetidos, usar o workflow TXT em vez de reescrever sequências longas. Durante debug, manter o ficheiro pequeno e reprodutivel.

Guardar exemplos reprodutíveis

Ao reportar um bug, incluir:

- versão exata do ATC;
- modo de precisão;
- modo angular;
- comandos introduzidos;

- resultado esperado;
- resultado obtido;
- se `verbose resolution` estava ativo.

Respeitar o âmbito do ATC

O ATC não substitui ferramentas profissionais de validação específicas de dominio. Em decisoes criticas de engenharia, finanças ou ciência, comparar resultados importantes com metodos independentes.

Evitar comportamento não documentado

Não construir workflows dependentes de comportamento que não esteja descrito no guia, nas notas de release ou nos testes. Os internos do parser e do solver podem evoluir mantendo o comportamento documentado.